

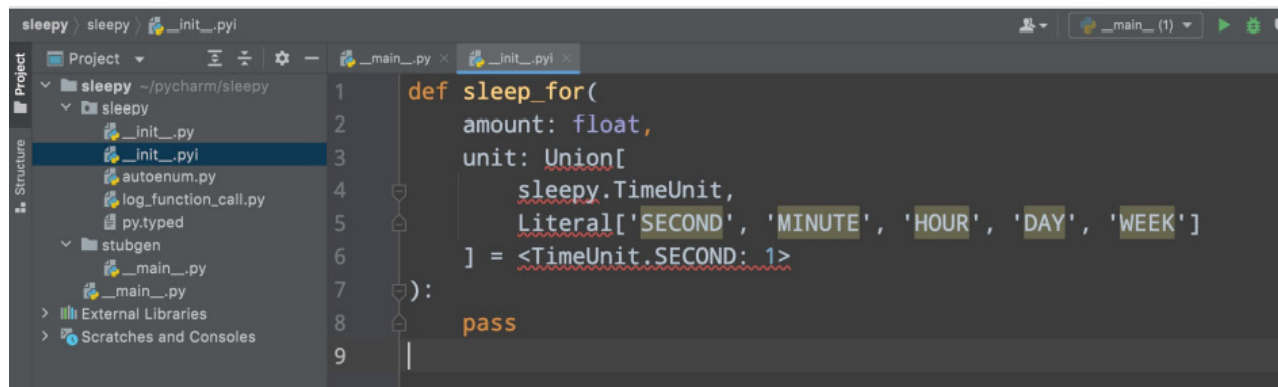


Figure 9: Result of refactoring the code

source code from the path as if it was a module called *name*. We'll see how this works later.

Next, we need to create a special function that generates the stub files so that this function will accept the module object. This is a simplified version but will only generate stub files for functions. We will take our module and traverse all the attributes of our modules. If the value that is stored in the attribute is a function, and this function was defined in a specified module (some modules might be imported from somewhere else), then we only want to create the stub files to create the functions that are defined in our module. We will write the keyword, the name of the function, the signature of the function, and an empty body. The only thing 'Function main' does is that it takes a path to a module and imports it, creates a stub file, and then writes this file into the file that ends with an 'I'. Now if we take a sleepy dot 'py' module, it will write the stub file to *sleepy.pyi* and we can run the code for

our sleepy module.

We want to import the sleepy module as a sleeper module, and it is located at *sleepy/youNeedThatFile*. We can get a long line of code by running it, but if we refactor the code we will see something like what is shown in Figure 9.

We now have the *sleep_for* function and a signature that is pretty much similar to what we expected but not quite. From this simple example we can see all the difficulties we will encounter while trying to take this approach.

This code was simple and did not generate a syntactically correct Python file. This was because there was a problem with the default value and the required files were also not imported for it to be a valid Python file. We did not import *literal* and *union*, or define

sleepy.TimeUnit. But since the IDE is clear, with a little more effort we can easily make this code work.

In this case, we used the Stubmaker tool. This tool accepts the name and path to the package and output directories to generate stub files. This allows us to get automatically generated stub files that are created after introspecting the runtime objects.

It is essential to understand the need for proper code analysis while working on a project. Try to seal as many loopholes as possible, so that problems like low cohesion are overcome. Developer time is valuable, and should be spent on improving the project rather than fixing routine problems. So, go for automatic stub generation to make their life easier too. **END** 🐧

By: Vladimir Losev

The author is head of the crowdsourcing tools development group at Toloka. This article is based on a tech talk he gave at OSI 2021. The complete talk can be viewed at <https://youtu.be/gvwKvco4aXo>.

Transcribed and curated by Laveesh Kocher

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource

ForYou

The latest from the Open Source world is here.

OpenSourceForU.com

Join the community at facebook.com/opensourceforu

Follow us on Twitter @OpenSourceForU